



UNIVERSIDAD ESTATAL DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE CIENCIAS DE DATOS E INTELIGENCIA ARTIFICIAL

CURSO:

CDDEIA-ELNO-4-2

AGENTE CONVERSACIONAL PARA ADMISIÓN Y NIVELACIÓN DE LA UG

ESTUDIANTES:

IVÁN ANDRÉS MOYA ASTUDILLO
MARCO ANTONIO MOYA RAMÍREZ

ASIGNATURA:

PROCESAMIENTO DEL LENGUAJE NATURAL

DOCENTE:

ING. CUENCA ORTEGA ANGEL

PERÍODO LECTIVO

Ciclo I 2026 – 2027

Contenido

Resumen del proyecto	3
Introducción al problema abordado	3
Fuentes de datos y estructura del proyecto	3
Arquitectura y funcionamiento del pipeline	4
Preprocesamiento de texto	4
Detección de intención: heurística + similitud coseno	5
Construcción de la respuesta	5
Demostración de funcionamiento	6
Evaluación cuantitativa del chatbot	6
Análisis de los errores encontrados	7
Justificación de las decisiones técnicas	8
a) TF-IDF con unigramas y bigramas frente a una bolsa de palabras simple	8
b) Similitud coseno para comparar la consulta contra el corpus	8
c) Preprocesamiento antes de vectorizar	8
d) Umbral de confianza (0.2) y manejo de fallback	8
e) Datos (JSON/CSV) separados del código	8
f) Heurística híbrida para la duración de carreras	9
Actualizaciones del proyecto (segunda versión del notebook)	9
g) Manejo de errores en la carga de datos	9
h) Extracción de entidades por expresiones regulares: cédula y fechas	9
i) Respuesta dinámica con fecha exacta de inscripción	10
j) Manejo de errores en el bucle principal de conversación	10
Conclusiones	11
Fuentes	11

Resumen del proyecto

Título: Agente Conversacional para Admisión y Nivelación de la UG.

Elaborado por: Iván Andrés Moya Astudillo y Marco Antonio Moya Ramírez.

El notebook `PLN_ProyectoP2.ipynb` implementa el diseño y desarrollo de un agente conversacional basado en técnicas clásicas de Procesamiento de Lenguaje Natural que, a partir de datos reales obtenidos de fuentes oficiales, identifica la intención del usuario y devuelve respuestas pertinentes a preguntas frecuentes sobre el proceso de admisión y nivelación de la Universidad de Guayaquil, manejando adecuadamente las consultas que el sistema no logra reconocer.

Introducción al problema abordado

Durante cada período de admisión, la Universidad de Guayaquil recibe un alto volumen de preguntas repetitivas por parte de los aspirantes: fechas de inscripción y postulación según el número de cédula, dónde y cuándo se rinde el examen de admisión, cómo funciona la asignación y aceptación de cupos, en qué consiste el curso de nivelación, requisitos y fechas de matrícula, y qué carreras existen dentro de cada bloque de conocimiento.

El notebook resuelve este problema implementando un chatbot que responde en español utilizando únicamente técnicas clásicas de PLN (vectorización TF-IDF y similitud coseno), sin recurrir a modelos de deep learning ni a generación abierta de lenguaje. Los datos de intenciones (`intents.json`) y de entidades del dominio (`entities.json`), así como el conjunto de pruebas (`test_queries.csv`), se descargan directamente desde un repositorio de GitHub del propio equipo, con respaldo en caché local para evitar descargas repetidas.

Fuentes de datos y estructura del proyecto

El notebook consume tres archivos externos, descargados y cacheados localmente mediante la función `load_data()`:

- `intents.json` — 21 intenciones, cada una con su lista de patrones (`utterances`) de ejemplo y sus respuestas asociadas.
- `entities.json` — diccionario de conocimiento del dominio: bloques de conocimiento de la UG (Salud, Administrativas, Educación, Ingeniería, Sociales, Básicas) y el listado de carreras con su bloque y duración en semestres.
- `test_queries.csv` — 30 consultas de prueba con su intención real (`intent_true`), usadas para evaluar el desempeño del chatbot.

Las 21 intenciones definidas en `intents.json` cubren tres grupos temáticos:

- Conversacionales: saludo, despedida, identidad_bot.

- Núcleo de admisión: `consulta_fecha_inscripcion_postulacion`, `consulta_examen_admision`, `consulta_temario_examen_admision`, `consulta_asignacion_cupos`, `consulta_aceptacion_cupo`, `consulta_segunda_tercera_postulacion`.
- Nivelación y carreras: `consulta_que_es_nivelacion`, `consulta_fechas_matricula_nivelacion`, `consulta_requisitos_matricula_nivelacion`, `consulta_retiro_nivelacion`, `consulta_inicio_clases_nivelacion`, `consulta_desarrollo_nivelacion`, `consulta_aprobacion_nivelacion`, `consulta_plataformas_virtuales`, `consultar_canales_contacto`, `consulta_carreras_bloque`, `consulta_duracion_carrera`, y la intención de reserva fallback.

Arquitectura y funcionamiento del pipeline

El notebook organiza el flujo de trabajo en funciones independientes, todas dentro de una única celda de código:

- `load_data()` / `load_json_from_url()` — carga los JSON desde disco si ya existen localmente, o los descarga desde GitHub y los guarda en caché.
- `preprocess(text)` — limpieza y normalización del texto de entrada.
- `extract_career(knowledge, user_input)` — detecta si el usuario mencionó el nombre de una carrera, comparando el texto preprocesado del mensaje contra el preprocesado de cada nombre en `entities.json`.
- `detect_intent(...)` — determina la intención del usuario combinando una heurística de reglas con similitud coseno sobre TF-IDF.
- `get_response(...)` — construye la respuesta final, con lógica adicional para los casos de duración de carrera y listado de carreras por bloque.
- `evaluate_chatbot(...)` — ejecuta el conjunto de prueba, calcula accuracy y F1-macro, y reporta los errores de clasificación.
- `main()` — orquesta la carga de datos, entrena el vectorizador TF-IDF, abre un bucle de conversación por consola y, al salir, corre la evaluación final.

Preprocesamiento de texto

La función `preprocess()` aplica, en este orden:

- Conversión a minúsculas.
- Eliminación de signos de puntuación y caracteres especiales mediante la expresión regular `[\^a-z0-9\s]`.
- Eliminación de tildes con `unidecode` (por ejemplo, "admisión" pasa a "admision").
- Tokenización con `nltk.word_tokenize` en español.
- Eliminación de stopwords en español (`nltk.corpus.stopwords`).
- Stemming con `SnowballStemmer('spanish')`, que reduce cada palabra a su raíz.

```

1 def detect_intent(knowledge, user_input, vectorizer, tags, X, umbral=0.2):
2     # Palabras clave para identificar consultas sobre duración o tiempo de estudio
3     duracion_keywords = ['dura', 'duracion', 'semestre', 'semestres', 'tiempo', 'anos', 'años']
4
5     # 1. Limpieza y preprocesamiento de la entrada del usuario
6     user_clean = preprocess(user_input)
7
8     # 2. Heurística: si detectamos una carrera y palabras de duración, forzamos la intención
9     carrera = extract_career(knowledge, user_input)
10    if carrera and any(k in user_clean for k in [preprocess(w) for w in duracion_keywords]):
11        return 'consulta_duracion_carrera', 1.0
12
13    # 3. Vectorización TF-IDF de la entrada
14    user_vec = vectorizer.transform([user_clean])
15
16    # 4. Similitud coseno contra todos los patrones conocidos
17    sims = cosine_similarity(user_vec, X).flatten()
18    best_idx = np.argmax(sims)
19
20    if sims[best_idx] >= umbral:
21        return tags[best_idx], sims[best_idx]
22    return 'fallback', 0.0

```

Captura del notebook: función detect_intent(), donde se aplica la heurística de duración de carrera y el umbral de similitud coseno.

Detección de intención: heurística + similitud coseno

detect_intent() no depende únicamente del vectorizador TF-IDF. Antes de calcular similitudes, revisa si el mensaje del usuario menciona una carrera (vía extract_career) junto con alguna palabra asociada a duración ("dura", "duracion", "semestre", "semestres", "tiempo", "anos", "años"); si ambas condiciones se cumplen, fuerza directamente la intención consulta_duracion_carrera con una confianza de 1.0, sin pasar por el cálculo de similitud. Esto evita que preguntas de duración con vocabulario muy corto (por ejemplo "cuanto dura medicina") no alcancen el umbral por tener pocas palabras en común con los patrones de entrenamiento.

Si esa heurística no aplica, el texto preprocesado se vectoriza con el TfidfVectorizer ya entrenado (ngram_range=(1,2), es decir, unigramas y bigramas) y se calcula cosine_similarity contra la matriz TF-IDF de todos los patrones del corpus. Se toma el índice de mayor similitud; si ese valor es mayor o igual al umbral (0.2 por defecto), se devuelve la etiqueta correspondiente junto con el puntaje; de lo contrario, se devuelve la intención fallback con similitud 0.0.

Construcción de la respuesta

get_response() busca la intención detectada dentro de intents_data y selecciona aleatoriamente una respuesta de su lista responses. Sobre esa base general aplica dos casos especiales:

- Si la intención es consulta_duracion_carrera y se identificó una carrera en el mensaje, la respuesta se reemplaza por una frase generada dinámicamente con el nombre de la carrera y su número de semestres, tomados de entities.json.
- Si la intención es consulta_carreras_bloque, el sistema busca coincidencias entre el mensaje del usuario y los nombres de los bloques definidos en entities.json; si encuentra uno, lista todas las carreras de ese bloque. Si no logra identificar un bloque específico, devuelve un mensaje genérico invitando a especificar bloque o carrera.

Demostración de funcionamiento

A continuación se muestran ejemplos de interacción reproducidos ejecutando el código del notebook con los datos reales descargados desde el repositorio de GitHub del proyecto (intents.json y entities.json). Cada respuesta incluye la línea de depuración [Debug: intención=..., similitud=...] que el propio main() imprime en cada turno de la conversación.

```
Tú: Hola
[Debug: intención=saludo, similitud=1.00]
Chatbot: ¡Hola! Soy el asistente virtual de la Universidad de Guayaquil. Pregúntame sobre admisión,
nivelación o carreras.

Tú: Cuando son las inscripciones
[Debug: intención=consulta_fecha_inscripcion_postulacion, similitud=1.00]
Chatbot: La creación de cuenta y selección de carreras se realiza según el último dígito de tu cédula:
- Dígitos 1 y 2: 13 de julio de 2026
- Dígitos 3 y 4: 14 de julio
- Dígitos 5 y 6: 15 de julio
- Dígitos 7 y 8: 16 de julio
- Dígitos 9 y 0: 17 de julio
- Todos los dígitos: desde el 18 hasta el 20 de julio de 2026.

Tú: Cuanto dura medicina
[Debug: intención=consulta_duracion_carrera, similitud=1.00]
Chatbot: Medicina dura 10 semestres.
```

Consultas de saludo, fecha de inscripción según cédula y duración de una carrera (heurística de duración activada con similitud forzada a 1.00).

```
Tú: Que carreras hay en el bloque de salud
[Debug: intención=consulta_carreras_bloque, similitud=0.88]
Chatbot: Carreras en Ciencias de la Salud:
- Enfermería
- Laboratorio Clínico
- Medicina
- Nutrición y Dietética
- Obstetricia
- Odontología
- Optometria
- Psicología
- Radiología e Imagenología
- Terapia Fisica
- Terapia Ocupacional
- Terapia de Lenguaje

Tú: Como me contacto con la universidad
[Debug: intención=consultar_canales_contacto, similitud=0.37]
Chatbot: Puedes seguir los canales oficiales: Facebook 'Admision UG' e Instagram 'Coordinacion de
Admision y Nivelacion' para comunicados.
También puedes revisar el portal https://admision.ug.edu.ec/admision/.

Tú: gracias, eso es todo
[Debug: intención=despedida, similitud=1.00]
Chatbot: Gracias por consultar. Estoy aquí cuando me necesites.
¡Mucha suerte en tu proceso! Hasta pronto.
```

Consulta de carreras por bloque de conocimiento, canal de contacto (similitud baja, 0.37) y despedida.

Se observa que la pregunta sobre canales de contacto obtiene una similitud relativamente baja (0.37), por estar redactada de forma distinta a los patrones de entrenamiento, aunque aún logra superar el umbral de 0.2 y clasificarse correctamente.

Evaluación cuantitativa del chatbot

El notebook incluye una función `evaluate_chatbot()` que corre las 30 consultas de `test_queries.csv` contra el pipeline de detección de intención, compara la etiqueta predicha con la etiqueta real (`intent_true`) y calcula dos métricas con `scikit-learn`: `accuracy_score` y `f1_score` (promedio macro). Los resultados detallados se exportan a `evaluation_results.csv`.

```

=== RESULTADOS DE EVALUACIÓN ===
Total de consultas evaluadas: 30
Accuracy (exactitud): 80.00%
F1-macro: 82.35%

=== ERRORES ENCONTRADOS ===
Consulta: 'mi cedula termina en 5 cuando me toca'
Real: consulta_fecha_inscripcion_postulacion | Predicho: consulta_examen_admision | Similitud: 0.37

Consulta: '¿donde rindo el examen de admision?'
Real: consulta_examen_admision | Predicho: consulta_temario_examen_admision | Similitud: 0.68

Consulta: 'horario de la prueba'
Real: consulta_examen_admision | Predicho: consulta_temario_examen_admision | Similitud: 0.41

Consulta: 'temario para medicina'
Real: consulta_temario_examen_admision | Predicho: consulta_duracion_carrera | Similitud: 1.00

Consulta: 'que es la nivelacion de carrera'
Real: consulta_que_es_nivelacion | Predicho: consulta_carreras_bloque | Similitud: 0.48

Consulta: 'duracion de ingenieria de software'
Real: consulta_duracion_carrera | Predicho: fallback | Similitud: 0.00

```

Salida real de `evaluate_chatbot()` ejecutada sobre las 30 consultas de prueba: 80.00% de accuracy y 82.35% de F1-macro.

Análisis de los errores encontrados

De las 30 consultas evaluadas, 6 fueron clasificadas incorrectamente:

- "mi cedula termina en 5 cuando me toca" → se esperaba `consulta_fecha_inscripcion_postulacion`, pero se predijo `consulta_examen_admision` (similitud 0.37). El mensaje no contiene las palabras clave de fecha o inscripción que dominan ese patrón, por lo que el vector queda más cerca de otra intención con vocabulario parcialmente compartido.
- "¿donde rindo el examen de admision?" → se esperaba `consulta_examen_admision`, pero se predijo `consulta_temario_examen_admision` (similitud 0.68). Ambas intenciones comparten el bigrama "examen admision", lo que genera confusión entre preguntas de lugar y de contenido del examen.
- "horario de la prueba" → mismo par de intenciones confundidas que el caso anterior, con una similitud aún más baja (0.41), evidenciando que "prueba" como sinónimo de "examen" no está bien representado en los patrones de entrenamiento.
- "temario para medicina" → se esperaba `consulta_temario_examen_admision`, pero al mencionar el nombre de una carrera ("medicina") sin palabras de duración, el sistema no activa la heurística de duración, sin embargo el vectorizador terminó favoreciendo `consulta_duracion_carrera` con similitud 1.00, mostrando una limitación de la heurística basada solo en palabras clave.
- "que es la nivelacion de carrera" → se esperaba `consulta_que_es_nivelacion`, pero se predijo `consulta_carreras_bloque` (similitud 0.48), posiblemente por la palabra "carrera" presente en ambos conjuntos de patrones.

- "duracion de ingenieria de software" → se esperaba consulta_duracion_carrera, pero cayó en fallback con similitud 0.00. Es probable que "ingenieria de software" no coincida exactamente con ningún nombre de carrera registrado en entities.json, por lo que extract_career no logra detectarla y la heurística de duración no se activa.

En conjunto, estos errores muestran que la principal fuente de confusión del modelo es el vocabulario compartido entre intenciones temáticamente cercanas (examen vs. temario del examen, nivelación vs. carreras por bloque), y que la heurística de extracción de carreras depende de una coincidencia textual exacta con la lista cerrada de nombres en entities.json.

Justificación de las decisiones técnicas

a) TF-IDF con unigramas y bigramas frente a una bolsa de palabras simple

El vectorizador se configura como `TfidfVectorizer(ngram_range=(1, 2))`. Además del conteo de unigramas, incorpora bigramas (pares de palabras consecutivas), lo que permite capturar expresiones como "curso nivelacion" o "examen admision" como unidades con más peso discriminativo que sus palabras sueltas. El factor IDF castiga términos muy frecuentes en el corpus (como "admision" o "universidad", presentes en casi todas las intenciones) y da más relevancia a los términos propios de pocas intenciones, mejorando la separación entre clases que comparten vocabulario general.

b) Similitud coseno para comparar la consulta contra el corpus

`detect_intent()` usa `cosine_similarity(user_vec, X)` en lugar de una métrica basada en distancia. La similitud coseno mide el ángulo entre vectores y no su magnitud, por lo que una consulta corta y un patrón más largo sobre el mismo tema pueden seguir teniendo una similitud alta si comparten palabras relevantes, sin que la diferencia de longitud entre ambos textos distorsione la comparación.

c) Preprocesamiento antes de vectorizar

El orden de `preprocess()` (minúsculas → limpieza de símbolos → eliminación de tildes → tokenización → eliminación de stopwords → stemming) reduce la variabilidad del vocabulario antes de construir el vector TF-IDF. Por ejemplo, "inscripción", "inscripcion" e "inscribirme" terminan compartiendo una raíz común gracias al stemming, de modo que el vectorizador las trata como manifestaciones de la misma palabra base en lugar de tres términos sin relación aparente.

d) Umbral de confianza (0.2) y manejo de fallback

El parámetro `umbral=0.2` en `detect_intent()` define el mínimo de similitud aceptable para que el sistema confíe en su predicción. Si la mejor coincidencia no alcanza ese valor, se devuelve la intención fallback con similitud 0.0, evitando que el chatbot responda con aparente certeza ante consultas totalmente fuera de dominio (por ejemplo, texto sin relación alguna con el corpus de patrones).

e) Datos (JSON/CSV) separados del código

Las intenciones, los patrones, las respuestas y el diccionario de entidades no están escritos dentro del código Python: se cargan dinámicamente desde `intents.json`, `entities.json` y `test_queries.csv`, alojados en un repositorio de GitHub externo al notebook. Esto permite editar o ampliar el contenido del chatbot (agregar una intención, corregir una respuesta, sumar una carrera nueva) sin tocar la lógica del programa, y reutilizar el mismo código para otro dominio simplemente cambiando los archivos de datos.

f) Heurística híbrida para la duración de carreras

Además del enfoque puramente estadístico basado en TF-IDF, el notebook incorpora una regla explícita (`extract_career` + palabras clave de duración) para resolver un tipo de consulta específico que el modelo de similitud, por sí solo, tendía a clasificar de forma menos confiable dado lo corto y variable de este tipo de preguntas ("cuanto dura X", "duracion de X"). Esta combinación de reglas y vectorización estadística es un patrón común en chatbots basados en PLN clásico, donde ciertas intenciones de alta frecuencia se resuelven con lógica determinista y el resto se delega al modelo estadístico general.

Actualizaciones del proyecto (segunda versión del notebook)

En una revisión posterior del notebook (`PLN_Proyecto_P2.ipynb`) se incorporaron cuatro mejoras sobre la versión inicial, orientadas a robustecer el manejo de errores y a enriquecer las respuestas del chatbot con información calculada a partir de entidades extraídas por expresiones regulares.

g) Manejo de errores en la carga de datos

La función `load_data()` ahora envuelve la lectura del archivo local y la descarga desde GitHub en un bloque `try/except` que captura `json.JSONDecodeError` y `requests.exceptions.RequestException`. Si el archivo local está corrupto o la descarga falla (por ejemplo, por falta de conexión), el programa imprime un mensaje explicativo y devuelve una estructura vacía en lugar de detenerse con un `traceback`, dejando que el resto del flujo continúe de forma controlada.

h) Extracción de entidades por expresiones regulares: cédula y fechas

Se añadieron cuatro funciones nuevas para la extracción de entidades del dominio mediante `regex`, complementando a `extract_career()`:

- `extract_cedula(user_input)` — busca un número de 10 dígitos consecutivos en el mensaje del usuario y devuelve tanto la cédula completa como su último dígito.
- `fecha_por_digito(ultimo_digito)` — aplica la regla oficial de la UG que asocia cada último dígito de cédula con una fecha concreta de creación de cuenta y selección de carrera (13 al 17 de julio de 2026, según el par de dígitos).
- `extract_fecha(user_input)` — reconoce fechas escritas en texto ("13 de julio de 2026") o en formato numérico ("13/07/2026", "13-07-2026") mediante dos expresiones regulares.
- `extract_entities(knowledge, user_input)` — función unificada que combina las anteriores junto con `extract_career()` y devuelve un único diccionario con todas las entidades detectadas en la consulta (carrera, cédula, último dígito, fecha de inscripción calculada y fecha en texto libre).

```

1 def extract_cedula(user_input):
2     match = re.search(r"\b\d{10}\b", user_input)
3     if match:
4         cedula = match.group()
5         ultimo_digito = int(cedula[-1])
6         return cedula, ultimo_digito
7     return None, None
8
9 def fecha_por_digito(ultimo_digito):
10    # Regla oficial: fecha de inscripción/postulación según el último dígito de la cédula
11    if ultimo_digito in (1, 2):
12        return "13 de julio de 2026"
13    elif ultimo_digito in (3, 4):
14        return "14 de julio de 2026"
15    ... (5-6, 7-8, 9-0)
16    return None
17
18 def extract_entities(knowledge, user_input):
19    # Función unificada de extracción de entidades por reglas/regex
20    cedula, ultimo_digito = extract_cedula(user_input)
21    return {
22        "carrera": extract_career(knowledge, user_input),
23        "cedula": cedula,
24        "ultimo_digito_cedula": ultimo_digito,
25        "fecha_inscripcion_por_cedula": fecha_por_digito(ultimo_digito) if ultimo_digito is not None
26    }
27    else None,
28    "fecha_texto": extract_fecha(user_input)
29    }

```

Captura del notebook: nuevas funciones `extract_cedula()`, `fecha_por_digito()` y `extract_entities()` para la extracción de entidades por regex.

i) Respuesta dinámica con fecha exacta de inscripción

`get_response()` incorpora un nuevo caso especial: cuando la intención detectada es `consulta_fecha_inscripcion_postulacion` y el usuario incluyó su número de cédula en el mensaje, el chatbot ya no devuelve la tabla completa de fechas por dígito, sino una respuesta personalizada que calcula directamente la fecha que le corresponde al usuario según el último dígito de su cédula.

```

Tú: Mi cedula es 0928656735 cuando me toca inscribirme
[Debug: intención=consulta_fecha_inscripcion_postulacion, similitud=0.29]
Chatbot: Según tu cédula (0928656735), terminada en 5, tu fecha para crear cuenta y seleccionar carrera
es el 15 de julio de 2026.

Tú:
Chatbot: Por favor escribe una consulta. No puedo procesar un mensaje vacío.

Tú: asdkjaskjd
[Debug: intención=fallback, similitud=0.00]
Chatbot: Disculpa, no entendí tu pregunta. ¿Podrías reformularla? Recuerda que puedo ayudarte con
admisión, nivelación y carreras de la UG.

```

Consulta con cédula real: la respuesta ahora calcula la fecha exacta de inscripción (15 de julio de 2026 para el dígito 5). También se muestran el manejo de una consulta vacía y el mensaje de fallback ante una entrada sin sentido.

Es importante notar que esta mejora no modifica el accuracy ni el F1-macro reportados por `evaluate_chatbot()`, ya que esa función evalúa únicamente `detect_intent()` (la clasificación de intención), mientras que la nueva lógica de fecha por cédula vive en `get_response()` (la generación del contenido de la respuesta). El efecto de este cambio es una respuesta más útil y precisa para el usuario final, no una mejora medible en las métricas de clasificación.

j) Manejo de errores en el bucle principal de conversación

Dentro de `main()`, cada turno de la conversación ahora se procesa dentro de un bloque `try/except`. Antes de intentar detectar la intención, se valida si la entrada del usuario está vacía o contiene solo espacios (`if not entrada or not entrada.strip()`), en cuyo caso se pide reformular sin ni siquiera invocar al vectorizador. Cualquier otro error inesperado durante el procesamiento es capturado por `except Exception as e`, que informa el problema y permite que el ciclo de conversación continúe en el siguiente turno en lugar de terminar abruptamente.

Conclusiones

El agente conversacional implementado en el notebook alcanzó un 80.00% de accuracy y un 82.35% de F1-macro sobre el conjunto de 30 consultas de prueba, usando únicamente técnicas clásicas de PLN (TF-IDF, similitud coseno y una heurística de reglas para duración de carreras). Estas métricas se mantienen igual en la segunda versión del notebook, ya que las mejoras incorporadas (manejo de errores y fecha exacta por cédula) no inciden sobre la etapa de clasificación de intención.

La principal fuente de error observada sigue siendo la confusión entre intenciones temáticamente cercanas que comparten vocabulario (examen de admisión vs. temario del examen; nivelación vs. carreras por bloque), así como los casos en que `extract_career` no reconoce el nombre completo de una carrera si el usuario la escribe de forma distinta a como está registrada en `entities.json`. La segunda versión sí resuelve, a nivel de calidad de respuesta, el caso de la consulta con cédula ("mi cedula termina en 5 cuando me toca"): aunque la intención pueda no clasificarse siempre de forma exacta, cuando sí se identifica correctamente la intención de fecha de inscripción, ahora la respuesta entrega el dato preciso en lugar de la tabla completa.

Como trabajo futuro se podría ampliar el número de patrones por intención (especialmente para examen de admisión y su temario), añadir sinónimos y variantes de los nombres de carreras en `entities.json` para robustecer `extract_career`, evaluar un umbral distinto por intención en lugar de un umbral global de 0.2, y extender la lógica de `extract_entities()` para que sus resultados (`fecha_texto`, `cédula`) se aprovechen también en otras intenciones que podrían beneficiarse de respuestas personalizadas.

Fuentes

- Proceso de Admisión: <https://admision.ug.edu.ec/admision/>
- Curso de Nivelación: <https://admision.ug.edu.ec/nivelacion/>
- Calendario Académico: <https://admision.ug.edu.ec/calendario/>
- Carreras universitarias: <https://admision.ug.edu.ec/>
- Repositorio en GitHub con archivos auxiliares: https://github.com/lvn5739/Proyecto_PLN_P2